

Mission 2 of 4 — Architecture Foundation

Suggested time: ~15 minutes | Deliverable: Architecture Version 1

With the technology choices from Mission 1 confirmed, it's time to turn them into a working system design. NimbusCart is ready to see its first architecture diagram.

Requirements Recap

- Your Mission 1 selections (Frontend, Backend/Compute, Database, Auth, and Notifications/Messaging) are now locked in as the foundation of the system.
- NimbusCart still needs the same core flow: a seller or buyer logs in, browses or manages listings, and completes actions that trigger order and notification events.

Task: Build Architecture Version 1

- Place each component you chose in Mission 1 onto the Architecture Workspace canvas.
- Draw the connections that represent how a request actually travels — for example, a buyer's request reaching the Frontend, then the Backend API, then the Database and Auth service.
- Show where the Messaging/Notification component fits — typically triggered by an event (e.g. an order being placed) rather than sitting directly in the main request path.
- Label every component clearly with the technology you selected, so the flow is readable without extra explanation.

Guidance

- Think in terms of one concrete action (e.g. "a buyer places an order") and trace it step by step through your components.
- A connection should only exist if there's a real interaction behind it — avoid connecting components just because they're both on the canvas.
- Favor a clean, readable layout over a decorative one. Clarity is what the jury is scoring.

Deliverable

A completed Architecture Version 1 diagram in the Architecture Workspace, showing your chosen components and the logical connections between them.

Evaluation Focus: Software Architecture Design • Logical Component Integration

■ Hints — Mission 2 – Architecture Foundation

- Trace one real user action (e.g. "a buyer places an order") through your components, in order. Let that path shape your layout.
- Every connection you draw should represent an actual interaction — avoid connecting components just to fill space.
- Simplicity reads as clarity to a jury. A clean, well-labelled diagram beats a busy one.

■ Hints — General

- There is no single correct architecture. The jury is evaluating your reasoning and clarity, not whether you matched a hidden answer key.
- Re-read the client requirements and business constraints before opening the Technology Inventory — it's easy to pick a popular-sounding option that doesn't actually fit the brief.
- The 60 minutes are split across four missions. Pace yourself; don't spend so long perfecting Mission 1 that later missions get rushed.
- Every deliverable asks for a short written justification somewhere — keep a running note of *why* you chose each component, in plain language.
- Cost and complexity are trade-offs, not penalties. A more expensive or more complex option can be the right call if the requirements justify it — say so.

■ Frontend framework — quick decision guidance

Supplementary context, not part of the official brief — useful for explaining your reasoning.

- **React / Next.js** — reach for this when the team already knows React, when you need the largest hiring/library ecosystem, or when you want SSR and static CDN export together. It's the default "safe" choice for a small team on a deadline.
- **Vue.js / Nuxt.js** — reach for this when the team is newer to frontend frameworks and wants a gentler learning curve, or when the project favors an opinionated, batteries-included structure over React's more DIY ecosystem.
- **Angular** — fits large, long-lived enterprise apps with big teams that want strict structure and built-in tooling out of the box; usually overkill for a small team's pilot launch.
- **SvelteKit** — fits when bundle size and raw runtime performance matter most and the team is comfortable with a smaller (but growing) ecosystem.

■ Glossary — terms used on this page

Term	What it means
SPA (Single Page App)	A frontend built as one page that updates its content dynamically, instead of reloading a new page for every screen.
SSR (Server-Side Rendering)	Generating a webpage's HTML on the server before sending it to the browser, rather than building it in the browser.

Monolithic Architecture	A design where the UI, business logic, and other concerns run together as one deployable unit, rather than as separate services.
React / Next.js	A component-based frontend library (React) plus a framework built on it (Next.js) that adds routing, SSR, and CDN-friendly static export.
Vue.js / Nuxt.js	A component-based frontend framework (Vue) plus a framework built on it (Nuxt) with a gentler learning curve and a more opinionated, batteries-included setup than React.
Angular	A full-featured, opinionated frontend framework maintained by Google, with built-in tooling for large, structured enterprise applications.
SvelteKit	A frontend framework built on Svelte, which compiles components into small, highly optimized vanilla JS at build time instead of shipping a runtime framework to the browser.
Auto-Scaling Compute	Backend compute capacity (VMs or containers) that automatically increases or decreases based on traffic load.
Horizontal Scaling	Handling more load by adding more instances of a service, rather than making a single instance bigger.
Load Balancer	A component that distributes incoming requests across multiple backend instances, so no single instance is overwhelmed.
API Gateway	A managed entry point that routes, authenticates, and rate-limits requests before they reach backend services.
Message Queue	A component that holds messages between systems so tasks can be processed asynchronously instead of immediately, blocking the user.
OAuth2 / OIDC	Standard protocols that let an app confirm a user's identity and permissions, often via a third-party identity provider, without the app storing passwords itself.
JWT (JSON Web Token)	A compact, signed token used to represent a user's identity and permissions between requests.
Relational Database (RDBMS)	A structured database organized into tables with defined relationships, typically queried using SQL.
NoSQL Document Store	A database that stores data as flexible, schema-less documents rather than fixed rows and columns.
Read Replica	A copy of a database used to handle read queries, so the primary database is freed up mainly for writes.